

Lesson 1.9: Core Concepts

Controller, Service, Module & Dependency Injection (DI)

- **Khóa học:** NestJS Practical Masterclass: Zero to Hero
- **Dự án thực chiến:** Real-Time Social Chat Application
- **Thời lượng dự kiến:** ~12 - 15 phút



Bộ Ba Kiến Trúc Cốt Lõi



Controller

Đón nhận HTTP Requests (GET, POST), validate dữ liệu & điều hướng đường dẫn API.



Service (Provider)

Chứa toàn bộ Business Logic, tính toán nghiệp vụ & truy vấn CSDL.



Module

Đóng gói và gom nhóm các Controllers & Services thành khối tính năng độc lập.



Dependency Injection (DI) Là Gì?

❌ Lập Trình Truyền Thống

Controller tự khởi tạo Service với từ khóa `new` :

```
class UsersController {  
  private service = new UsersService();  
}
```

⚠️ Khó unit test, phụ thuộc chặt chẽ (Tight Coupling).

✅ NestJS Dependency Injection

Nhận Service từ IoC Container qua `Constructor` :

```
class UsersController {  
  constructor(  
    private service: UsersService  
  ) {}  
}
```

⚡ Dễ dàng Mock test, quản lý Singleton tự động.



Khởi Tạo Nhanh Với Nest CLI

Sử dụng **Nest CLI** để sinh cấu trúc chuẩn trong 1 click:

1. Tạo Users Module

```
pnpm nest g module users
```

2. Tạo Users Service (bỏ file test .spec)

```
pnpm nest g service users --no-spec
```

3. Tạo Users Controller (bỏ file test .spec)

```
pnpm nest g controller users --no-spec
```

💡 **Mẹo:** Nest CLI tự động khai báo Controller & Service vào mảng `controllers` và `providers` của `UsersModule`.



UserService (@Injectable)

Chứa logic nghiệp vụ và truy vấn dữ liệu:

```
import { Injectable } from '@nestjs/common';

export interface User { id: number; name: string; email: string; }

@Injectable()
export class UserService {
  private users: User[] = [
    { id: 1, name: 'Alice', email: 'alice@example.com' },
  ];

  findAll(): User[] {
    return this.users;
  }
}
```



UsersController (@Controller)

Tiêm `UserService` qua Constructor và khai báo Route:

```
import { Controller, Get } from '@nestjs/common';
import { User, UserService } from '../users.service';

@Controller('users')
export class UsersController {
  constructor(private readonly userService: UserService) {}

  @Get()
  getAllUsers(): User[] {
    return this.userService.findAll();
  }
}
```

● Lỗi Thường Gặp: UnknownElementException

```
ERROR [ExceptionHandler] Nest can't resolve dependencies of UsersController (?).  
Please make sure that the argument UserService at index [0] is available in UsersModule context.
```

🔍 Nguyên Nhân & Khắc Phục:

- **Nguyên nhân:** Quên khai báo `UserService` vào mảng `providers` trong `UsersModule`.
- **Khắc phục:** Thêm `UserService` vào `providers: [UserService]` của tệp `users.module.ts`.

Tổng Kết Bài Học

- ● **Controller:** Đón nhận & phản hồi HTTP Request (`@Controller`).
 - ● **Service:** Xử lý toàn bộ logic nghiệp vụ (`@Injectable`).
 - ● **Module:** Khối gom nhóm tính năng (`@Module`).
 - ● **DI & IoC:** NestJS tự tạo Singleton Instance và tiêm vào Constructor.
- 👉 **Bài tiếp theo:** Lesson 1.10 - Configuration & Environment Variables (`@nestjs/config`)